25 Patterns of Time Complexity

🎯***Goal:*** To solve every pattern of time complexity that you will ever face in Data Structure problems.

PATTERN 1:

```
void printNumbers(int n) {
    for (int i = 1; i <= n; i++) {
        System.out.print(i + " ");
    }
}
```

Time Complexity Analysis:

- The loop runs from 1 to n, iterating n times.
- Each iteration takes O(1), so overall complexity is **O(n)**.

PATTERN 2:

```
void printPairs(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            System.out.println("(" + i + ", " + j + ")");
        }
    }
}
```

Time Complexity Analysis:

- Outer loop runs n times. Inner loop runs n times for each outer iteration.
- Total iterations = n×n=O(n2)n \times n = O(n^2)n×n=O(n2).

PATTERN 3:

```
void cubicLoop(int n) {
    for (int i = 1; i <= n; i++) {       // Outer loop (Runs n times)
        for (int j = 1; j <= n; j++) {   // Middle loop (Runs n times)
            for (int k = 1; k <= n; k++) { // Inner loop (Runs n times)
                System.out.println("CodeN");
            }
        }
    }
}
```

CODE UNISHCHAL

Explanation:

- Outer loop (`i`): Runs O(n) times.
- Middle loop (`j`): Runs O(n) times for each `i`.
  Inner loop (`k`): Runs O(n) times for each `j`.

  = O(n)×O(n)×O(n)=O(n^3)

PATTERN 4:

```
void printPowersOfTwo(int n) {
    for (int i = 1; i <= n; i *= 2) {
        System.out.print(i + " ");
    }
}
```

Time Complexity Analysis:

- `i` starts at `1` and doubles each time.
- Number of times the loop runs is log base 2 of n.
  So complexity is O(logn).

PATTERN 5:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.println(i + " " + j);
    }
}
```

Time Complexity Analysis:

Outer loop runs → **n times**

Inner loop runs → **i times** for each `i`

Total executions of inner statement:

1+2+3+...+n

We know:

1+2+3+...+n =n(n+1)/2
Ignore constants:

≈n^2/2

Drop constant in Big-O:

Final Answer: O(n^2)

PATTERN 6:

```
void printHalving(int n) {
    for (int i = n; i > 0; i /= 2) {
        System.out.print(i + " ");
    }
}
```

Time Complexity Analysis:

- `i` starts at `n` and is halved each time.
- Number of times it runs is log base 2 of n.
- **REMEMBER**: If this (i/=2) divide is happening by any other number then the base will be that number instead of 2.
- So complexity is O(logn).

PATTERN 7:

```
void printWithStep(int n) {
    for (int i = 1; i <= n; i += 3) {
        System.out.print(i + " ");
    }
}
```

Time Complexity Analysis:

- The loop runs roughly **n/3** times.
- As constants are ignored, the complexity is **O(n)**.

CODEWITHNISHCHAL

PATTERN 8:

```
void printLogPairs(int n) {
    for (int i = 1; i <= n; i *= 2) {
        for (int j = n; j > 0; j /= 2) {
            System.out.println("(" + i + ", " + j + ")");
        }
    }
}
```

Time Complexity Analysis:

- Outer loop runs O(logn) times.
- Inner loop runs O(logn) times.
- Total complexity: O(log^2 n).

PATTERN 9:

```
void printGrowingLoop(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j *= 2) {
            System.out.println("(" + i + ", " + j + ")");
        }
    }
}
```

Time Complexity Analysis:

- Outer loop runs O(n) times.
- Inner loop runs O(logn) times.
  Total complexity: O(nlogn).

PATTERN 10:

```
void printDecreasingSteps(int n) {
    for (int i = n; i > 0; i /= 2) {
        for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
}
```

Time Complexity Analysis:

- Outer loop runs **O(logn)** times.

- Inner loop runs **O(i)** times, summing up to **O(n).**
  Total complexity:O(n).

PATTERN 11:

```
void complexLoop(int n) {
    for (int i = 1; i <= n; i++) {  // Runs O(n) times
        for (int j = 1; j <= n; j++) {  // Runs O(n) times
            for (int k = 1; k <= n; k++) {  // Runs O(n) times
                int x = 1;
                while (x <= k) {  // x doubles each time
                    System.out.println("CodeN");
                    x *= 2;
                }
            }
        }
    }
}
```

Explanation:

1. **Outer loop (`i`): Runs O(n) times.**
2. **Second loop (`j`): Runs O(n) times.**
3. **Third loop (`k`): Runs O(n) times.**
4. **Innermost loop (`while` loop on `x`):**
   - **Starts from `1` and doubles (`x *= 2`) until it exceeds `k`.**
   - **This loop runs O(log k) times.**

   T(n) = O(n*n*n*log n)=O(n^3 log n)

PATTERN 12:

```
void specialLoop(int n) {
    int i = 1, s = 1;
    while (s <= n) {
        System.out.println("CodeN");
        i++;
        s = s + i;
    }
}
```

Explanation:

- **The loop starts with `i = 1` and `s = 1`.**
  In each iteration:

DEVITHNISHCHAL

- **`i` increases by `1` (`i++`).**
  `s` increases by the updated value of `i` (`s = s + i`).

The loop continues while `s ≤ n`.

Tracking the Iterations:

Iteration	i	s (after update)
1	2	1 + 2 = 3
2	3	3 + 3 = 6
3	4	6 + 4 = 10
4	5	10 + 5 = 15
...	...	...
k	k+1	Sum(1 to k+1) ≥ n

The stopping condition is when:

$$s=1+2+3+...+k\geq n$$

The sum of the first k natural numbers is:

$$s=k(k+1)/2$$

Setting it equal to n to find k:

$$k(k+1)/2\approx n$$

Solving for k: K^2≈2n so k will be **√2n**

Time complexity is O(√n).

PATTERN 13:

```
void specialLoop(int n) {
    for (int i = 1; i * i <= n; i++) {
        System.out.println("CodeN");
    }
}
```

For i^2 <=n or i<=√n: for such patterns TC will be O(√n) only. (check the sheet)

Number of loop calls here is √n.

CODEWITHNISHCHAL

PATTERN 14:

```
void nestedLoop(int n) {
    int i, j, k;
    for (i = 1; i <= n; i++) {        // Outer loop (Runs n times)
        for (j = 1; j <= i; j++) {    // Middle loop (Runs i times)
            for (k = 1; k <= 11; k++) { // Innermost loop (Runs 11 times)
                System.out.println("CodeN");
            }
        }
    }
}
```

Explanation:

- **Outer loop** (`i`): Runs from `1` to `n` → **`n` iterations**.
- **Middle loop** (`j`): Runs from `1` to `i` → **`i` iterations for each `i`**.
- **Innermost loop** (`k`): Always runs **11 times**, irrespective of `i` or `j`.

Total Iterations Calculation:

- For a given `i`, the **middle loop** runs `i` times.
- The **innermost loop** runs `11` times for each middle loop iteration.
- Total executions of `"CodeN"`:

For i=1, j will execute 1 times and k will execute 11 times = 1*11 times total

For i=2, j will execute 2 times and k will execute 2*11 times = 2*11 times total

For i=3, j will execute 3 times and k will execute 3*11 times = 3*11 times total

So on. 1*11+ (2*11) + (3*11) + …. = 11(1+2+3+4 + ….)

= 11(n(n+1)/2) = 11(n^2+n)/2 = **O(n^2)**

PATTERN 15:

```
void nestedLoop(int n, int q) {
    int i, j, k;
    for (i = q; i <= n; i++) {        // Outer loop (Runs from q to n)
        for (j = 1; j <= i * i; j++) {  // Middle loop (Runs i² times)
            for (k = 1; k <= n / 2; k++) { // Innermost loop (Runs n/2 times)
                System.out.println("CodeN");
            }
        }
    }
}
```

CODEWITHNISHCHAL

Explanation:

- Outer loop (`i`): Runs from `q` to `n` → `(n - q + 1)` iterations.
- Middle loop (`j`): Runs i^2 times for each value of `i`.
- Innermost loop (`k`): Runs `n/2` times (constant for all iterations).

For i=1, j will execute 1 times and k will execute n/2*1 times = n/2*1 times total

For i=2, j will execute 4 times and k will execute (n/2)*4 times = (n/2)*4 times total

For i=3, j will execute 9 times and k will execute (n/2)*9 times = (n/2)*9 times total so on…

= (n/2)*1 + (n/2)*4 + (n/2)*9 + ….. =n/2 (1+4+9+.....)

=n/2(1^2+ 2^2 + 3^2 + …) = n/2(n(n+1)(2n+1)/6) = O(n^4)

PATTERN 16:

```
void nestedLoop(int n) {
    int i, j, k;
    for (i = n / 2; i <= n; i++) {       // Outer loop (Runs from n/2 to n)
        for (j = 1; j <= n / 2; j++) {    // Middle loop (Runs n/2 times)
            for (k = 1; k <= n; k = k * 2) { // Innermost loop (Runs log(n) times)
                System.out.println("CodeN");
            }
        }
    }
}
```

Explanation:

- Outer loop (`i`): Runs from `n/2` to `n` → `(n - n/2) = n/2` iterations.
- Middle loop (`j`): Runs from `1` to `n/2` → `n/2` iterations.
- Innermost loop (`k`): Starts from `1` and doubles (`k = k * 2`) each time until it reaches `n`. This means it runs $\log_2(n)$ times in each iteration.

  = n/2 * n/2 * $\log_2(n)$ = O(n^2 $\log_2(n)$)

CODEWITHSHCHAL

PATTERN 17:

```
void nestedLoop(int n) {
    int i, j, k;
    for (i = n / 2; i <= n; i += 2 * i) { // Outer loop (Starts from n/2, incre
        for (j = 1; j <= n; j++) {        // Middle loop (Runs n times)
            for (k = 1; k <= n; k *= 2) { // Innermost loop (Runs log n times)
                System.out.println("CodeN");
            }
        }
    }
}
```

Explanation:

- Outer loop (`i`):
  - Start: `n/2 and` Next: `n/2 + 2*(n/2) = 3n/2` → already **> n**
  - So the outer loop runs **only once**. -> runs O(1) times.
- Middle loop (`j`):
  - Runs from `1` to `n`. So O(n) iterations.
- Innermost loop (`k`):
  - Starts at `1`, doubles (`k = k * 2`) until reaching `n`.
  - Runs O(log n) times.

= O(1)×O(n)×O(logn)=O(nlog n)

PATTERN 18:

```
void specialLoop(int n) {
    while (n > 1) {
        n = n / 2;
    }
}
```

Explanation:

- The loop starts with `n` and keeps dividing `n` by 2 until `n` becomes `≤ 1`.
- This means the number of iterations corresponds to how many times `n` can be divided by `2` before it reaches `1`.

Iteration	n (Before Update)	n (After Update)
1	n	n/2
2	n/2	n/4
3	n/4	n/8
...	...	...
k	n/2^k	1 (Loop stops)

CODE WITH NISHCHAL

The stopping condition is: $n/2k \leq 1$, Solving for k: $2k \geq n$

Taking logarithm on both sides: $k \geq \log2(n)$

Time Complexity Analysis:

- The loop runs approximately **$\log_2(n)$** times.
- Time complexity is **O(log n)**.

PATTERN 19:

```
void specialLoop(int n) {
    for (int i = 1; i <= n; i++) {       // Outer loop (Runs n times)
        for (int j = 1; j <= n; j += i) { // Inner loop (Increments by i)
            System.out.println("CodeN");
        }
    }
}
```

Explanation:

- Outer loop (`i`): Runs from 1 to n, meaning O(n) iterations.
- Inner loop (`j`):
    - Starts at 1, increments by `i` in each step (`j += i`).
    - Runs O(n / i) times for each `i`.

For i=1, j will execute n times and k will execute n times = n times total

For i=2, j will execute n times and k will execute n/2 times = n/2 times total

For i=3, j will execute 9 times and k will execute n/3 times = n/3 times total so on…

$$= n(1+\tfrac{1}{2} + \tfrac{1}{3} + \tfrac{1}{4} + \ldots\ldots + 1/n)$$

T(n)=n(log n)

PATTERN 20:

```
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

This function follows the recurrence relation:

T(n)=$T(n-1)+T(n-2)+O(1)$ **(checkout this pattern in my guide for faster calculation in interview)**

where:

CODEWITHNISHCHAL

- $T(n-1)$ is the recursive call for $(n-1)$
- $T(n-2)$ is the recursive call for $(n-2)$.
- $O(1)$represents the constant time taken for the addition operation.

For recursive programs time complexity depends on 2 things:

1. **Recursive Call Tree Growth**
2. **Number of Function Calls**

HERE:

Recursive Call Tree Growth:
Each function call spawns two recursive calls:

- fibonacci(n−1)
- fibonacci(n−2)

This forms a **binary recursion tree**. The depth of the tree is approximately O(n),but since each level almost doubles the number of calls, the total number of function calls follows an **exponential growth**.

Number of Function Calls:
The number of calls at each level follows the Fibonacci sequence itself:

- At level 0: 1 call (fibonacci(n).
- At level 1: 2 calls (fibonacci(n−1) and fibonacci(n−2)
- At level 2: 4 calls
  At level 3: 8 calls
  ... and so on.

Since this forms a **binary tree of depth O(n)**, the number of calls approximately follows:

$T(n) \approx 2^n$

Substitution Method (for above problem)

Using repeated expansion, we express T(n) in terms of smaller subproblems.

$$T(n)=T(n-1)+T(n-2)+O(1)$$

Expanding $T(n-1)$:

$$T(n-1)=T(n-2)+T(n-3)+O(1)T(n-1)$$

Substituting this into the original equation:

$$T(n)=(T(n-2)+T(n-3)+O(1))+T(n-2)+O(1)$$

$$T(n)=(T(n-2)+T(n-3)+O(1))+T(n-2)+O(1)$$

CODEWITHNISHCHAL

$$T(n)=2T(n-2)+T(n-3)+2O(1)$$

Continuing this process, the number of recursive calls doubles at each step, leading to:

$T(n) \approx O(2^n)$

Example Execution for n = 5

```
              fibonacci(5)

             /          \

       fibonacci(4)       fibonacci(3)

      /        \       /        \

fibonacci(3)  fibonacci(2) fibonacci(2) fibonacci(1)

/     \      /   \     /   \

f(2)   f(1)    f(1)  f(0)   f(1)  f(0)
```

Total function calls: 15

Many redundant calls (e.g., `fibonacci(3)` is computed twice).

PATTERN 21:

```java
void permute(String str, String answer) {
    if (str.length() == 0) {
        System.out.println(answer);
        return;
    }
    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        String left = str.substring(0, i);
        String right = str.substring(i + 1);
        permute(left + right, answer + ch);
    }
}
```

Time Complexity Analysis:

- Generates all n! permutations.
- Total complexity: **O(n!)**.

VITHUSHCHAL

PATTERN 22:

```
int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n - 1);
}
```

Recurrence Relation:

$T(n)=T(n-1)+O(1)$

Time Complexity Analysis:

- The function calls itself n times.
- Each call does O(1) work.
- Total complexity: O(n).
- **(checkout this pattern in my guide for faster calculation in interview)**

Substitution method:

Step 1: Expand the Recurrence

We repeatedly expand $T(n-1)$ $\quad T(n)=T(n-1)+O(1)$

Expanding $T(n-1)$ $\quad T(n-1)=T(n-2)+O(1)$

Substituting back: $\quad T(n)=(T(n-2)+O(1))+O(1)$

$T(n)=T(n-2)+2O(1)$

Expanding $T(n-2)$ $\quad T(n-2)=T(n-3)+O(1)$

Substituting again:

$$T(n)=(T(n-3)+O(1))+2O(1)$$

$$T(n)=T(n-3)+3O(1)$$

Following this pattern, after k expansions:

$$T(n)=T(n-k)+kO(1)$$

Now

$$T(n)=T(1)+(n-1)O(1)$$

Since $T(1)$ is a base case and takes constant time $O(1)$:

CODEWITHNISHCHAL

$T(n)=O(1)+(n-1)O(1)$

$T(n)=O(n)$

PATTERN 23: (checkout this pattern in my guide for faster calculation in interview)

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}
```

Recurrence Relation:

$T(n)=T(n-1)+O(1)$

Time Complexity Analysis:

- The function calls itself n times.
- Each call performs O(1) work.
- Total complexity: O(n).

PATTERN 24:

```
void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}
```

Recurrence Relation:
NOTE: THESE PATTERNS YOU CAN ALSO SOLVE BY MASTER THEOREM.
I am using the substitution method here.

$T(n)=2T(n/2)+O(n)$

Time Complexity Analysis:

- **Divide step: O(1)**
- **Merge step: O(n)**
- **Recurrence expands to: O(nlogn)**

Calculation: Step 1: Expand the recurrence

Expanding for a few levels:

$$T(n)=2T(n/2)+O(n)$$

$$T(n/2)=2T(n/4)+O(n/2)$$

Substituting:

$$T(n)=2[2T(n/4)+O(n/2)]+O(n)$$

$$=4T(n/4)+2O(n/2)+O(n)$$

Repeating for k steps:

$$T(n)=2kT(n/2k)+kO(n)$$

Substitute k and you will get $T(n)=O(n \log n)$

PATTERN 25:

```
void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Recurrence Relation:
QuickSort is a recursive call here and if you see this is happening 2 times but with low to pi-1 and pi-1 to high. So it is for n/2 input for both the times.

$$T(n)=2T(n/2)+O(n) \Rightarrow O(n\log n)$$

Keep Coding, Grind Hard and **Connect With Me** on Social Media.

Watch Full time complexity playlist:

L:1 Time Complexity | Introduction to Time Complexity | CodeWithNishchal

SHOW SOME LOVE BY FOLLOWING **CodeWithNishchal:**

- **Instagram:** https://www.instagram.com/codewithnishchal/
- **Book a 1:1 on Topmate:** https://topmate.io/nishchal_muradia
- **LinkedIn:** https://www.linkedin.com/in/nishchal-muradia/
- **YouTube Videos :** https://www.youtube.com/@CodeWithNishchal
- **Watch My Latest Time Complexity Series: Time Complexity**
- Telegram: https://t.me/codewithnishchal

CODEWITHNISHCHAL

CODEWITHNISHCHAL